

University of Victoria



ECE556 PROJECT REPORT

Learn To Battle Reinforcement Learning Approach For Robocode Competition

by

Quoc Dung Van (V01073195)

Summer 2025

Instructor: Hong-Chuan Yang

Abstract

Reinforcement learning is a popular technique to help an autonomous program learn optimal action through interacting with the environment. Robocode is a Java-based programming platform where autonomous tank agents battle in a 2D environment. This project aims to apply Reinforcement Learning to train a Robocode tank agent that is capable of navigating, aiming, and firing effectively in 1vs1 matches. By formulating Robocode as a Markov Decision Process (MDP) problem and applying Double Deep Q-Networks algorithm, the tank agent can learn optimal actions and achieve a 100% win rate after 2500 episodes. Although the developed agent can compete with simple tank strategies, it struggles against more advanced strategies. Other methods such as policy gradient algorithms should be explored to handle continuous action spaces. In addition, spatial feature extraction using a convolutional approach could provide richer information to improve the agent's decision-making.

Introduction

Reinforcement Learning (RL) is an area of machine learning where an agent learns to make decisions by interacting with an environment to maximize the total reward. Similar to how humans learn by experiencing consequences, agents learn through its trial and error process. Using RL to control agents in a dynamic environment has been extensively studied in many research works. One of the most influential benchmarks showcasing RL in games is the Arcade Learning Environment (ALE), which provides dozens of Atari 2600 games [2] for training and evaluation. Some of these games are shown in Figure 1. ALE presents agents with a variety of tasks that are not trivial to complete. However, by applying RL the agents can easily outperform human players in many of these games .



Figure 1: Five Atari 2600 games: Pong, breakout, Space Invaders, Seaquest, Beam Rider.

Robocode [1], created in 2000 by Mathew A. Nelson, is a Java-based programming game in which players program virtual tanks to battle in a 2D arena. It has since become widely used

in programming and AI courses, where learners build autonomous agents to apply algorithmic ideas in the simulated environment.

In Robocode, instead of manually controlling the tank, players implement logic for movement, targeting, firing, and radar scanning. The platform provides information about the player's tank, such as its energy, position, and the orientations of the body, gun, and radar. In addition, an event-driven API reports occurrences such as detecting an enemy, bullet hits, and wall collisions. Based on this information, the programmed agent decides which actions to take.

The platform simulates all players' actions and the corresponding physics at each tick interval, including turn rates, bullet speeds, and tank positions. This tick-based environment makes Robocode a suitable candidate to be formulated as a Markov Decision Process (MDP), enabling the application of reinforcement learning (RL) techniques to train the tank to learn optimal actions.

In this project, we will explore the ability of reinforcement learning in building an agent for Robocode and aim to develop a simple yet effective agent for 1v1 matches.



Figure 2: Robocode platform with multiple tanks competing each other .

MDP Formulation

In this project, we explore the capability of reinforcement learning (RL) in building a smart Robocode agent. Although Robocode supports multiplayer battles as illustrated in Figure 2, we restrict our scope to single-player, 1v1 matches. The agent will compete against a relatively simple, fixed tank strategy rather than an advanced one. Under this assumption, our goal is to develop a robot that can effectively navigate the battlefield, rotate its gun toward the target, and fire accurately, eventually increasing the overall winning rate.

Environment and Agent

The environment is the Robocode simulation engine, which manages the 2D battlefield, physics, and game rules. It progresses in discrete time steps (ticks), during which it updates the positions and states of all robots, bullets, and other game elements. The environment provides the agent with Java API to retrieve state information, and event functions which are triggered if there is collision or bullet hit.

The agent is our learning-based tank (robot), implemented as a Java class that interacts with the Robocode API. At each tick, it receives observations from the environment and uses its learned policy to decide on actions such as moving, turning the gun, or firing bullets. The agent's objective is to perform actions that maximize its long-term cumulative reward, which is associated with dealing damage to the enemy tank, avoiding the enemy's bullets and ultimately winning the match.

State Space

The state space captures all relevant information available for the agent to make a decision at each timestep. It includes self-state variables and enemy-related variables. For self-state variables, we have:

- Energy: the health metric of the tank.
- Tank location (x, y): 2D location of the tank.
- Tank / Gun / Radar direction: direction of tank, gun, and its radar.
- Gun heat: the temperature of the gun. The tank is only allowed to fire when gun heat is 0.
- Velocity: the current velocity of the tank.
- Travel distance remaining: the remaining distance that the tank needs to travel.

- Gun turn remaining: the remaining angle that the gun direction needs to turn.

For enemy-related variables, we have:

- Distance to enemy: travel distance to the enemy.
- Angle of enemy: angle between robot's direction and enemy.

We then have a total of 13 continuous variables, making an infinite state space.

Action Space

In Robocode, the tank could perform multiple actions with different param values. In this project, for simplicity I selected a limited set of actions, so that we have a finite action space. There are total of 8 discretized actions, including:

1. Move forward or backward 90 units distance.
2. Turn the body left or right and run with 50 units distance.
3. Aim the gun at the enemy.
4. Fire bullet with different power levels (1, 2, or 3).

Although we only use a limited set of finite actions, these actions are sufficient for the tank to achieve our goals, such as avoiding enemy bullets, aiming the gun at the enemy, and firing at the enemy.

Reward Design

The primary objective of our Robocode agent is to survive and defeat the enemy tank by avoiding unnecessary energy loss (get hit by enemy's bullet, hit the wall) and maximizing damage dealt. To guide the learning process, we designed our reward as follows:

- Penalties:
 - Hit wall: -5
 - Hit by enemy bullet: -5
 - Fire when gun temperature is overheated: -5
 - Lose round: -100
- Rewards:
 - Aim and fire in the correct direction: +5
 - Hit enemy with bullet: +5

- Win round: +100

This reward structure encourages the agent to fire accurately, avoid movements that result in collisions, and prioritize survival until the end of the round. Immediate feedback for both correct and incorrect actions helps to guide optimal actions, while the large terminal rewards and penalties encourage behaviors that lead to victory and discourage those that result in defeat.

Transition Dynamics

Robocode operates in a tick-based simulation. At each timestep, based on the actions of both the agent and the enemy tank, the environment transitions from the previous state to the next state. While the agent's own state information is deterministic and can be computed directly from the physics of the game, the enemy's state appears stochastic from the agent's perspective, since it cannot predict which action the opponent will take. As a result, the full transition probability is unknown and cannot be explicitly defined.

In general, the state transition is defined as $(s_t, a_t, r_{t+1}, s_{t+1})$ where s_t is current state at timestep t , a_t is the selected action for that time step. This action produces immediate reward r_{t+1} and lead to the next state s_{t+1} . We cannot fully describe all transition dynamics in this project; however, we examine several key transitions to illustrate how state changes occur in the Robocode environment.

The first example is the transition when the tank decides to perform turn and move action as illustrated in Figure 3. At time step t , the tank takes action "turnRightAndRun". At timestep $t+1$, the simulation will update the state by changing the location x , y , as well as the remaining distance and changing robot direction. The immediate reward for this action is 0.

The physics simulation is complicated but deterministic. Based on the tank direction and the travel distance for that time step, it can calculate the *delta x* and *delta y*, which will be used to update x , y location value. Similarly, the robot direction (heading) can also be calculated for next time step.

It should be noted that an action does not complete its full travel distance or turn to a fixed angle immediately; instead, the state is updated gradually, and the remaining portion of the action is recorded in the variables "distanceRemain" and "turnRemain". For example, at each timestep the maximum travel distance is 8 units and the maximum turn angle is 4 degrees. These values

decrease incrementally at every timestep as the agent continues to perform the corresponding actions.

	t	t+1
energy	97	97
x	209.97	212.08
y	325.45	317.73
heading	160.72	164.72
...	...	...
distanceRemain	0	42
turnRemain	0	86
Reward	+0	+0

Moving action will update the location and direction state

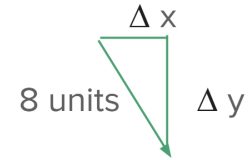


Figure 3: state transition when agent perform “turnRightAndRun”

The next example illustrates how the firing action is rewarded, as shown in Figure 4. At timestep t , the variables enemyBearing (the angle between the agent and the enemy) and gunHeading (the orientation of the gun) indicate whether the tank’s gun is aligned with the enemy. In addition, the gun temperature (gunHeat) is equal to 0, which allows the tank to fire. If the agent executes the “fire” action with a power of 2, it receives an immediate reward of +5, and the next state at timestep $t + 1$ is updated accordingly. In this case, the gun temperature increases to 2, reflecting the energy cost of firing with power of 2. Other state informations are also updated by physics simulation.

	t	t+1
energy	97	97
gunHeading	222.72	222.72
gunHeat	0	2
enemyBearing	200	180
enemyDistance	50	46

Reward +0 +5

Reward when fire
at the enemy
(correct direction)

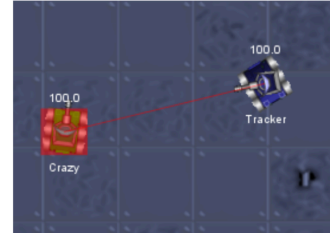


Figure 4: state transition when agent perform “fire” at the enemy

The final example, shown in Figure 5, illustrates how the agent is penalized when attempting to fire while the gun temperature is still above zero. At timestep t , the gun temperature (gunHeat) is 1.9, but the agent decides to execute the “fire” action. Since firing is not permitted while the gun is overheated, this action is discouraged and results in a penalty of -5. Afterward, the gun temperature continues to decrease by 0.1 units at each timestep as it cools down.

	t	t+1
energy	97	97
gunHeading	222.72	222.72
gunHeat	1.9	1.8
enemyBearing	200	180
enemyDistance	50	46

Reward +0 -5

Penalize when fire at
wrong direction, or the
gun is overheated

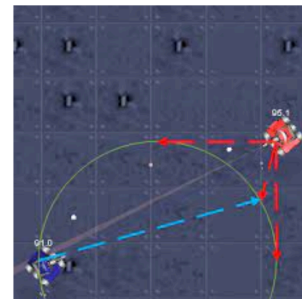


Figure 5: state transition when agent performs “fire” action with overheated gun.

Problem Classification

Episodic Task

In Robocode, a player's tank competes against other tanks over multiple rounds. Each round terminates when the player's tank either defeats all opponents or is destroyed. At the end of a round, the environment resets to its initial configuration, and the agent begins a new episode with a completely new state. Therefore, each round can be modeled as an episode in this reinforcement learning problem.

Stochastic Transition

The Robocode platform uses deterministic physics for movement and combat mechanics, including position updates, turning rates, bullet speeds, and collision handling. As a result, the evolution of the agent's own state can be predicted precisely given its current state and chosen action. However, the opponent's state depends on its internal logic and decision-making, which are unknown to the agent and may vary over time. Therefore, from the agent's perspective, this unknown property makes the overall state transition stochastic and the transition probability cannot be known in advance.

State and Action Space

Based on our MDP formulation, the state space consists of 13 variables, each taking values from a continuous range. For the action space, we have simplified control to a set of 8 discrete actions, as described in the previous section. In summary, the problem can be characterized as having a continuous state space and a finite, discrete action space.

Stationary Environment

Robocode platform has fixed rules, so its dynamics and reward function remain unchanged for every episode. With the assumption that the other tank's logic does not evolve throughout the battles, we can say the environment is stationary.

Solution

Algorithm Selection

Given Robocode's problem classification described in the above section, we selected the Double Deep Q-Network (Double DQN) [3] as our reinforcement learning algorithm for this project. The reasons for this choice are:

- As Robocode has a stochastic transition with unknown transition model, we cannot apply Dynamic Programming techniques to find the optimal action value, as the transition probability is impractical to know in advance. Model-free methods such as Tabular Q-learning or value function approximation are more suitable.
- For problems with a continuous state space, we cannot use traditional tabular Q-learning as it requires a discrete state space. We could discretize each variable in the state space into many bins, but this could lead to a very large number of state-action table entries. Furthermore, by discretizing the state space, detailed information available to the agent would be lost, which could negatively affect the decision-making process. Therefore, we chose to use value function approximation as the main technique for our tank agent.
- The relationship between the state space and the action-value function is complex and cannot be modeled effectively using a simple approach such as a linear relationship. To address non-linear relationships, deep neural networks are a good approach to approximate the action-value function. Therefore, a Deep Q-Network (DQN) will be used.
- Standard DQN tends to overestimate Q-values because it uses the same network for action selection and value evaluation. By using Double DQN, we can make the learning more stable by decoupling these roles, using one network for action selection and another for value evaluation.
- Policy-gradient algorithms are also good candidates to solve these kinds of problems, as they can handle continuous action spaces, but they are generally more complex to tune. For our goal of building simple yet effective agents, a discrete action space is sufficient. Therefore, in the scope of this project, we will use Double DQN for its simplicity and efficiency, and leave PPO as future work when we aim to build more advanced robots.

Double DQN algorithm

The Double DQN algorithm [3] is an extension of the standard Deep Q-Network, which is designed to reduce overestimation bias in Q-value estimates. In standard DQN, the same network is used to both select and evaluate the best action for the next state. Double DQN introduces 2 type of networks:

- The online network (Q Network): used to choose the next action
- The target network (Target Q Network): used to evaluate the chosen action

Similarly to DQN, Double DQN also uses Replay Memory, which stores the past experiences (state, action, reward, next state) for uniform random sampling and breaking the correlation between consecutive samples.

The full workflow is illustrated in Figure 6.

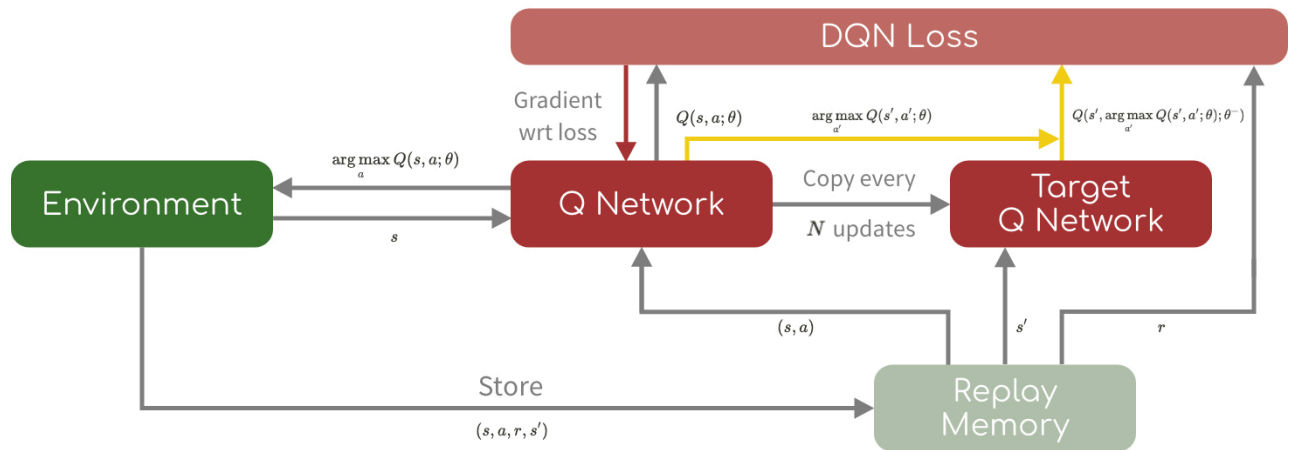


Figure 6: Double DQN workflow

Double DQN workflow details:

1. At the beginning, Q Network $Q(s, a; \theta)$ is initialized with random weight θ . Also, the Target Q Network's weight model is created with the same weight $\theta^- = \theta$ as the online Q Network.
2. At every tick, our tank's agent sends state, reward information and requests for action to perform. The ϵ -greedy algorithm is used to select an action that balances exploration (choosing a random action) and exploitation (choosing the action with the highest estimated Q-value $\operatorname{argmax}_a Q(s, a; \theta)$). The epsilon value is decay at a specific rate for each timestep.
3. The state, action, reward, next state information (s, a, r, s') is recorded in Replay Memory at each timestep.
4. Concurrently, a mini-batch from the replay memory is sampled and computed Double DQN target action value y . This value is then used to update the online Q Network parameters θ by minimizing the mean-squared error (loss function) between y and $Q(s, a; \theta)$.

$$\text{Loss function : } L(\theta) = \mathbb{E}_{(s,a,r,s')} \left[(y - Q(s, a; \theta))^2 \right]$$

$$\text{Target action value } y = r_{t+1} + \gamma Q \left(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a'; \theta), \theta^- \right)$$

5. Every N steps, the target Q Network's model θ^- is updated with online Q Network's model θ
6. The process is updated until convergence is reached or we reach a predefined number of episodes.

The complete pseudocode for this workflow is presented in Algorithm 1 below.

Algorithm 1 Double Deep Q-Network (Double DQN)

- 1: **Input:** learning rate α , discount factor γ , exploration rate ϵ , target update frequency N , replay buffer capacity N , batch size B
- 2: Initialize online network $Q(s, a; \theta)$ with random weights θ
- 3: Initialize target network $Q(s, a; \theta^-)$ with weights $\theta^- \leftarrow \theta$
- 4: Initialize empty replay buffer $\mathcal{D}$
- 5: **for** episode = 1 to M **do**
- 6: Initialize state s_0
- 7: **for** $t = 0$ to T **do**
- 8: With probability ϵ , select random action a_t
- 9: Otherwise, select $a_t = \arg \max_a Q(s_t, a; \theta)$
- 10: Execute a_t in environment, observe r_{t+1} and s_{t+1}
- 11: Store $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{D}$
- 12: Sample mini-batch of B transitions (s_j, a_j, r_j, s'_j) from $\mathcal{D}$
- 13: For each sample (s_j, a_j, r_j, s'_j) :
- 14: $a'_{\max} \leftarrow \arg \max_a Q(s'_j, a; \theta)$ ▷ Action selection using online net
- 15: $y_j \leftarrow r_j + \gamma Q(s'_j, a'_{\max}; \theta^-)$ ▷ Action evaluation using target net
- 16: Update θ by minimizing:

$$L(\theta) = \frac{1}{B} \sum_{j=1}^B (y_j - Q(s_j, a_j; \theta))^2$$

- 17: **if** $t \bmod N = 0$ **then**
 - 18: Update target network: $\theta^- \leftarrow \theta$
 - 19: **end if**
 - 20: $s_t \leftarrow s_{t+1}$
 - 21: **if** s_t is terminal **then**
 - 22: **break**
 - 23: **end if**
 - 24: **end for**
 - 25: **end for**
-

Implementation and Result

Robocode is a Java-based platform in which each tank agent is implemented as a Java class. While Java offers high execution speed, it is less commonly used than Python for developing artificial intelligence applications due to the supported machine learning libraries. To leverage the strengths of Python AI libraries, the tank agent in this project was implemented in Java as a proxy, serving as an interface between the Robocode environment and an external reinforcement learning agent implemented in Python. This design allows Robocode to handle simulation while Python manages the learning process. The overall architecture is illustrated in Figure 7 below.

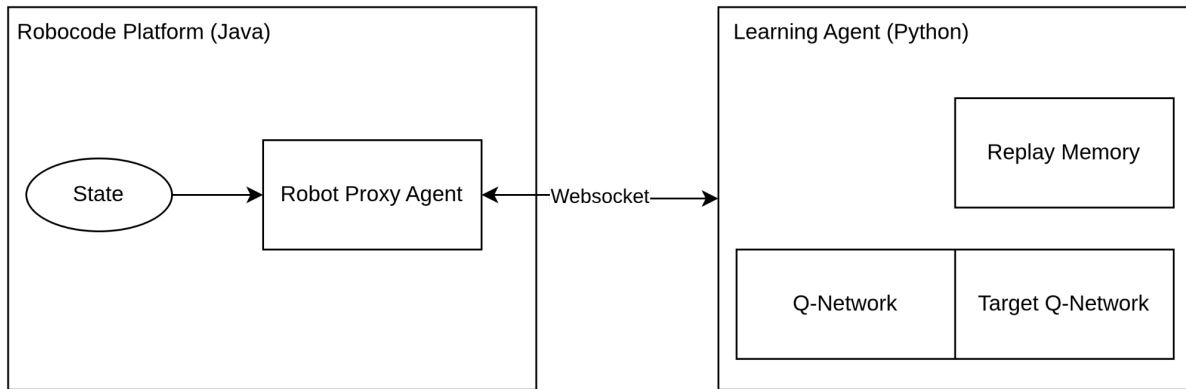


Figure 7: Architecture of reinforcement learning agent for Robocode

Robot Proxy Agent

Robot Proxy Agent is Java-based component within the Robocode platform that acts as an intermediary between the Robocode platform and the external reinforcement learning agent implemented in Python. Its primary role includes:

- At each timestep, the Robot Proxy Agent collects all relevant state information, including both the tank's own status and the detected enemy's information.
- This data is then serialized and transmitted to the learning agent via a WebSocket connection.
- Upon receiving the next action chosen by the learning agent, the proxy executes the corresponding action code in Java, allowing the Robocode platform to simulate the chosen action for that timestep.

In addition to the proxy implementation, the Robocode environment was modified to facilitate the learning process. The security management configuration was disabled to permit network communication with external programs. Furthermore, the default execution time limit per timestep (100 ms) was increased to ensure that the agent has sufficient time to perform the learning computations.

Reinforcement Learning Agent

The reinforcement learning agent is implemented as a WebSocket server and manages the state and training process of the Double DQN algorithm. At the beginning of each round, the tank proxy agent establishes a client connection to the server, thereby initiating a new episode. At each timestep, the proxy agent sends the current state information to the server and requests an action. The learning agent selects an action using the ϵ -greedy algorithm, as described in the previous section, and returns it to the proxy agent. Simultaneously, the learning process is carried out by sampling mini-batches from the Replay Memory and updating the weights of the Q-Network according to the Double DQN workflow discussed earlier. There are several notable configurations applied during the implementation of the Double DQN algorithm, as follows:

- Each Q-network is implemented with two hidden layers, each containing 512 neurons.
- To reduce computational cost, the learning process is executed once every four steps.
- The Replay Memory is implemented as an array with a capacity of 10,000 transitions.
- Sampling batch size for learning is 64.
- The Target Q-Network is updated every 1,000 steps.
- The exploration rate in ϵ -greedy algorithm is initialized at 1.0 and decays by a factor of 0.99995 at each step.

*The implementation of the entire project can be accessed from the GitHub repository [5].

Result

Figure 8 shows how the reward evolves over the episodes, alongside the exploration rate plot. Initially, the tank selects actions randomly, resulting in frequent penalties for incorrect actions. At the start of training, the average reward is approximately -700 . After about 2,500 episodes, the reward improves steadily and converges at around 200. It shows that the agent has learned the right action and gained reward over time.

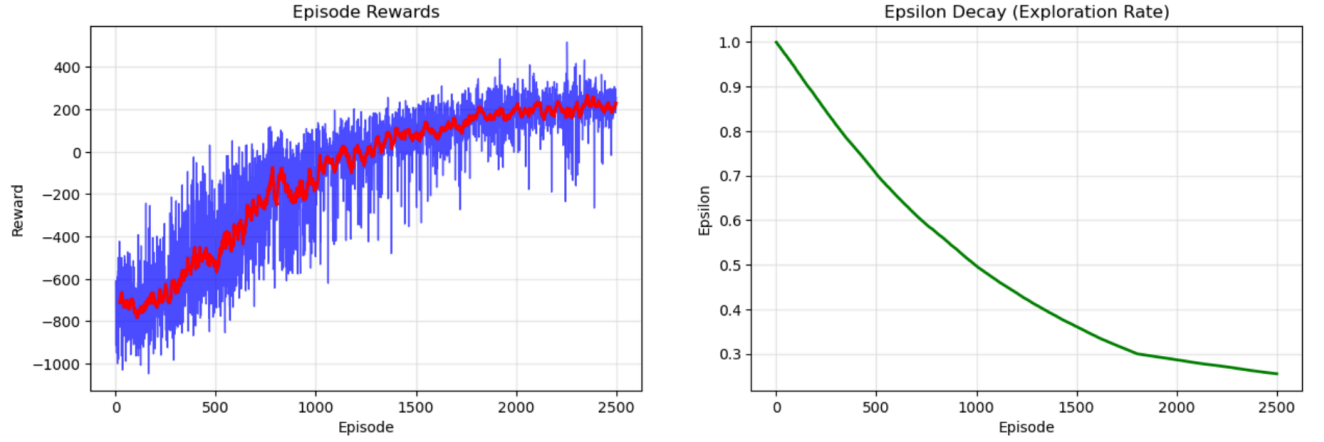


Figure 8: Reward and exploration rate over episodes

In addition to examining the learning process, we plot the number of losses over 20 episodes in Figure 9. The results show that the number of losses over 20 episodes decreases to nearly zero after approximately 2,500 episodes. This indicates that our tank agent has learned the correct actions and is able to win the game.

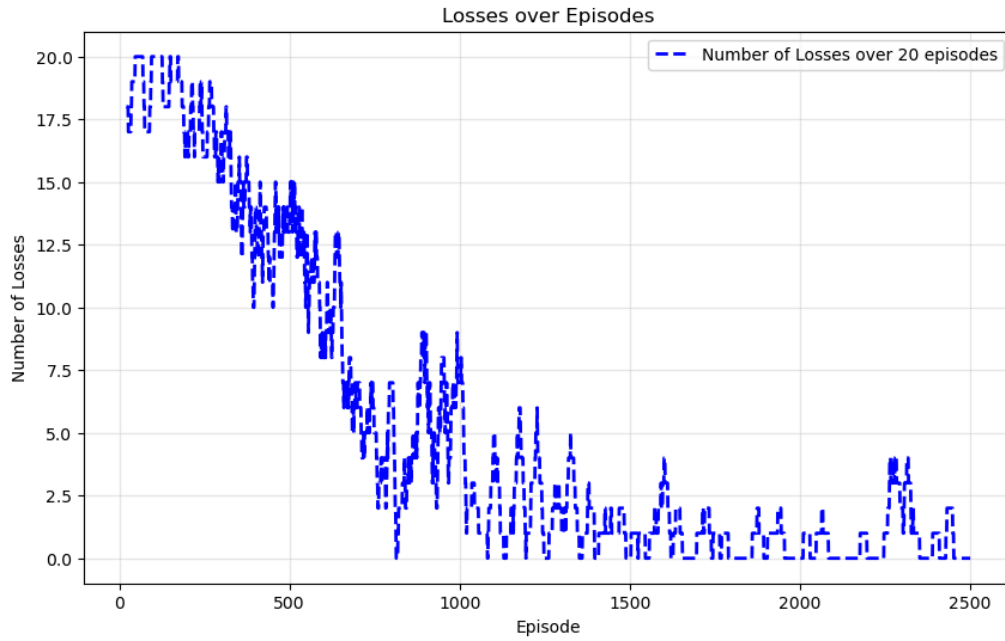


Figure 9: Number of losses over 20 episodes period.

Finally, we examined the loss function value. Figure 10 shows the training loss over the episodes. The results indicate that the average loss decreases significantly and nearly converges

after around 1,200 episodes. This reduction in loss suggests that the Q-network is learning a stable mapping from state-action pairs to their corresponding value estimates. As the loss converges, the updates to the network weights become smaller, indicating that the learned value function is approaching an optimal or near-optimal policy for the given environment. The stability observed after convergence also implies that the agent has gained a consistent understanding of which actions yield higher long-term rewards, leading to more reliable decision-making during gameplay.

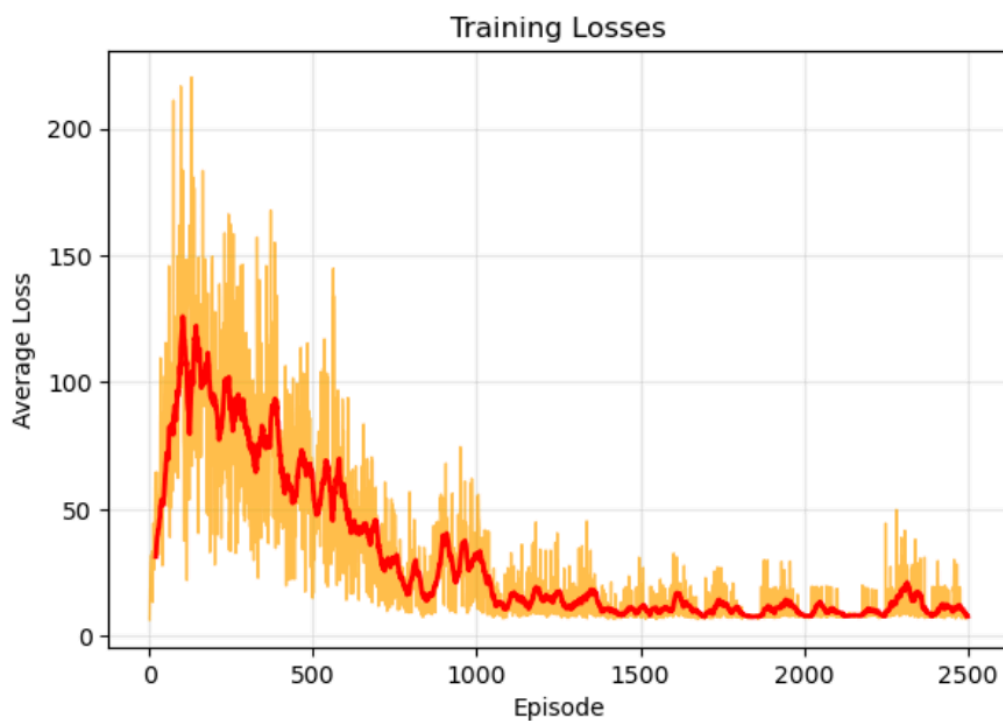


Figure 10: training losses over episodes

Conclusion

In this project, we successfully implemented a reinforcement learning agent for the Robocode platform using the Double Deep Q-Network algorithm. By designing an appropriate state and action space, shaping the reward function, and integrating a Java-based proxy agent with a Python learning agent, we are able to train a simple yet effective tank agent that can compete in the battlefield. Experimental results demonstrated that the agent's performance improved steadily over training, with rewards increasing from negative values to stable positive values, and win rates approaching 100% against the enemy tank after 2,500 episodes.

During the implementation of our tank agent, we found there are areas for improvement. While the current agent can compete successfully against simple logic tanks, it fails to consistently win against more advanced tanks that exhibit complex movement patterns. To address this limitation, the action space could be expanded by adding more discrete action, enabling finer control over the robot. Another promising direction is to apply policy-gradient (PPO) algorithms [4], which are capable of handling continuous action spaces and could improve performance in this dynamic environment. Additionally, Robocode does not provide explicit bullet trajectory information, which could be valuable for learning evasive maneuvers. This limitation could be solved by incorporating a sequence of battlefield snapshots and using a Convolutional Neural Network (CNN) to extract spatial information. Such an approach could capture useful information, including bullet trajectories and enemy movement patterns, allowing the agent to make better decisions.

References

- [1] RoboWiki, “Welcome to the RoboWiki,” *Robocode Wiki*, retrieved August 15, 2025, from https://robowiki.net/wiki/Main_Page
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing Atari with Deep Reinforcement Learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [3] H. van Hasselt, A. Guez, and D. Silver, “Deep Reinforcement Learning with Double Q-learning,” *arXiv preprint arXiv:1509.06461*, 2015.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] dustinvan910, “robocode,” *GitHub repository*, accessed August 15, 2025. [Online]. Available: <https://github.com/dustinvan910/robocode>